Classes and Objects Revision - Week 6

Category C Problems M1-M5 | Java solutions with verified output

M1. Library Inventory Management

Task: Replace three parallel arrays with a BookInventory class (title, author, copiesAvailable), a constructor, and printEntry(). Four objects are stored in a BookInventory[] and printed in a loop.

Solution (Main.java)

```
class BookInventory {
    String title;
    String author;
    int copiesAvailable;

    BookInventory(String title, String author, int copiesAvailable) {
        this.title = title;
        this.author = author;
        this.copiesAvailable = copiesAvailable;
    }

    void printEntry() {
        System.out.println(title + " by " + author + " - " + copiesAvailable + " copies
                available");
    }
}

public class Main {
    public static void main(String[] args) {
        BookInventory[] books = new BookInventory[4];
        books[0] = new BookInventory("Clean Code", "Robert C. Martin", 3);
        books[1] = new BookInventory("Effective Java", "Joshua Bloch", 5);
        books[2] = new BookInventory("Refactoring", "Martin Fowler", 0);
        books[3] = new BookInventory("Design Patterns", "GoF", 2);

        for (BookInventory b : books) {
            b.printEntry();
        }
    }
}
```

Output

```
Clean Code by Robert C. Martin - 3 copies available
Effective Java by Joshua Bloch - 5 copies available
Refactoring by Martin Fowler - 0 copies available
Design Patterns by GoF - 2 copies available
```

Note: Grouping the three related values into one object means a book's data can never drift out of sync, which was the cause of the broken recount.

M2. Payroll Salary Management

Task: PayrollAccount keeps basicSalary and bonus private. The constructor clamps a negative opening salary to 0 with a warning; creditBonus rejects amounts <= 0; deductTax rejects percent outside 0-100; getNetSalary() is the only (read-only) way to see the total.

Solution (Main.java)

```
class PayrollAccount {
    private double basicSalary;
    private double bonus;

    public PayrollAccount(double openingBasicSalary) {
        if (openingBasicSalary < 0) {
            System.out.println("Warning: negative salary not allowed. Starting at 0.");
            this.basicSalary = 0;
        } else {
            this.basicSalary = openingBasicSalary;
        }
        this.bonus = 0;
    }

    public void creditBonus(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid bonus amount: must be greater than 0.");
            return;
        }
        bonus += amount;
        System.out.println("Bonus credited: Rs " + amount);
    }

    public void deductTax(double percent) {
        if (percent < 0 || percent > 100) {
            System.out.println("Invalid tax percent: must be between 0 and 100.");
            return;
        }
        basicSalary -= basicSalary * percent / 100.0;
        System.out.println("Tax deducted: " + (percent % 1 == 0 ? String.valueOf((long) percent) :
                String.valueOf(percent)) + "%");
    }

    public double getNetSalary() {
        return basicSalary + bonus;
    }
}

public class Main {
    public static void main(String[] args) {
        PayrollAccount acc = new PayrollAccount(50000);
        acc.creditBonus(5000);
        acc.deductTax(10);
        System.out.println("Net salary: Rs " + acc.getNetSalary());
    }
}
```

Output

```
Bonus credited: Rs 5000.0
Tax deducted: 10%
Net salary: Rs 50000.0
```

Note: Encapsulation: there are no setters, so every change goes through a method that validates its input. Check: 50000 - 10% = 45000, plus bonus 5000 = 50000.

M3. Employee Profile Creation

Task: Two constructors: one for permanent employees (salary given, isIntern = false) and one for interns that chains with this(empId, empName, 0) and then sets isIntern = true.

Solution (Main.java)

```
class Employee {
    String empId;
    String empName;
    double salary;
    boolean isIntern;

    // Permanent employee
    public Employee(String empId, String empName, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.salary = salary;
        this.isIntern = false;
    }

    // Intern: chains to the 3-arg constructor, then marks as intern
    public Employee(String empId, String empName) {
        this(empId, empName, 0);
        this.isIntern = true;
    }

    void printProfile() {
        System.out.println(empId + " | " + empName + " | Rs " + salary + " | Intern: " +
                isIntern);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee permanent = new Employee("E-101", "Divya", 65000);
        Employee intern = new Employee("E-102", "Arjun");
        permanent.printProfile();
        intern.printProfile();
    }
}
```

Output

```
E-101 | Divya | Rs 65000.0 | Intern: false
E-102 | Arjun | Rs 0.0 | Intern: true
```

Note: Constructor chaining keeps the field setup in a single place instead of repeating it.

M4. Exam Hall Ticket Reference Management

Task: Two variables pointing at the same HallTicket object, a change made through one is visible through the other, and == compares references, not field values.

Solution (Main.java)

```
class HallTicket {
    String studentName;
    int seatNumber;

    HallTicket(String studentName, int seatNumber) {
        this.studentName = studentName;
        this.seatNumber = seatNumber;
    }
}

public class Main {
    public static void main(String[] args) {
        HallTicket priya = new HallTicket("Priya", 0);
        HallTicket copy = priya;          // same object, not a new one
        copy.seatNumber = 45;             // change through the second variable

        HallTicket separate = new HallTicket("Priya", 45);

        System.out.println("Priya's seatNumber (via first variable): " + priya.seatNumber);
        System.out.println("copy == priya: " + (copy == priya));
        System.out.println("separate == priya: " + (separate == priya));
    }
}
```

Output

```
Priya's seatNumber (via first variable): 45
copy == priya: true
separate == priya: false
```

Note: copy and priya hold the same reference, so == is true. separate is a different object with equal field values, so == is false.

M5. Employee and Company Information Management

Task: companyName and employeeCount are static, so they exist once per class and are shared. The constructor increments employeeCount. The static method printCompanyInfo() touches no instance field and is called through the class name.

Solution (Main.java)

```
class Employee {
    String empName;
    double salary;

    static String companyName = "Bright Horizon Technologies";
    static int employeeCount = 0;

    Employee(String empName, double salary) {
        this.empName = empName;
        this.salary = salary;
        employeeCount++;
    }

    static void printCompanyInfo() {
        System.out.println(companyName);
        System.out.println("Employees on record: " + employeeCount);
    }
}

public class Main {
    public static void main(String[] args) {
        new Employee("Divya", 65000);
        new Employee("Arjun", 30000);
        new Employee("Meera", 48000);

        Employee.printCompanyInfo();
    }
}
```

Output

```
Bright Horizon Technologies
Employees on record: 3
```

Note: Static members belong to the class, not to any object, so the company name is stored once rather than once per employee.